

Mickaël Laurent

Pierre Donat-Bouillud

Filip Křikava

Jan Vitek

Motivation

```
> x <- c(-1, 4, 9)
> if (x > 0) log(x) else 0
Error in if (x > 0) Log(x) else 0 : the condition has Length > 1
```

With types: the guard of `if` requires `lg1` (a scalar), `x > 0` has type `lg1`, rejects the `if` call

```
> scores <- factor(c("1", "2", "3"))
> wt <- c(0.2, 0.3, 0.5)
> w <- weighted.mean(scores, wt)
Warning message: In Ops.factor(x, w) : '*' not meaningful for factors
> w
[1] NA
```

With types: `weighted.mean` : `(x : dbl, w : dbl) -> dbl1` rejects the call, `scores` is `int<factor>`

```
> df <- data.frame(revenue = c(100, 200, 300))
> vat <- df$revenue * 0.21 # missing column, NULL * 1.2 is numeric(0)
> total <- sum(vat) # silently 0, error surfaces far from the typo
> total
[1] 0
```

With types: `df`: `{ revenue : dbl, ... }` has no field `revenue`, rejects `df$revenue` call

Code quality

- Lightweight specification and documentation

Safety

- Helps to catch bugs that tests miss (tests and type-checking are complementary)
- Static feedback: with types, less runtime errors!

Transparency

- R hides stuff: a value's storage type (`int` vs `dbl`), its shape (scalar vs vector vs matrix), its S3 classes, these are all invisible at the call site.
- **Types make the invisible bits explicit where needed!**

Goal

Types for R

- Syntax and semantics for R types
- Find the sweet-spot between precision and ergonomics

Type checking

- Static type checks - in an editor or command line, before code runs
- Runtime type checks - for cases that cannot be statically checked
- No more `stopifnot(is.character(file) && length(file) == 1)`

Type inference

- Infer types automatically so user only annotate when needed

Types for R

Atomic vectors

Vector types (`vec`): `raw | lg1 | int | dbl | clx | chr`
Scalar types (`vec1`): `raw1 | lg11 | int1 | dbl1 | clx1 | chr1`
Singleton types: `42 | "hello world" | tt | ff | ...`

Lists

List of (any number of) double vectors: `{ dbl }`
Exactly two named elements: `{ x : dbl, y : dbl }`
Two named elements and anything else: `{ x : dbl, y : dbl, any }`

S3 classes

Exactly this class: `<data.frame>`
At least this class: `<data.frame ...>`
- *can decorate any type* `{ id : int, name : chr }<data.frame ...>`
- *any boolean combination* `<matrix, ~array ...>`
- *for simplicity, types do not capture the order of classes*

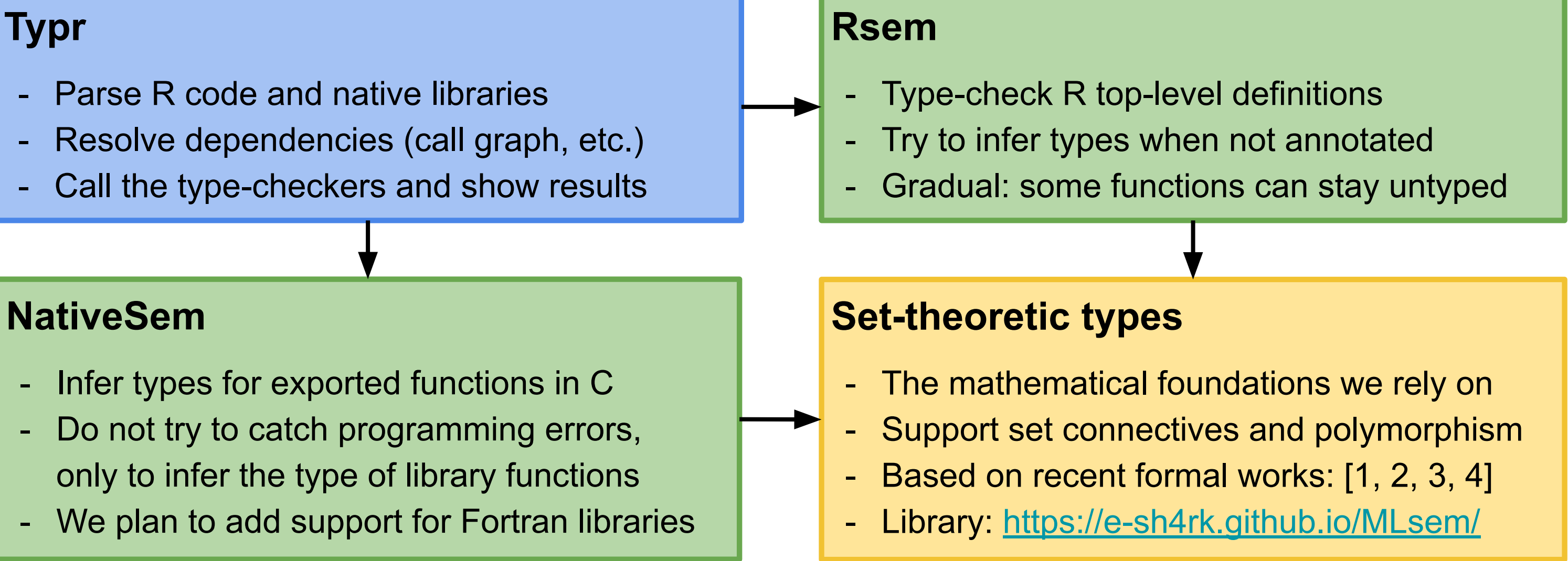
Functions

Types for function parameters and return type: `(x: dbl, y: int) -> dbl`
Arrow of unions: `(x: dbl | int | chr) -> (dbl | chr)`
Intersection of arrows: `((x: dbl) -> (dbl)) & ((x: int) -> (dbl)) & ((x: chr) -> (chr))`

Types are sets

Set-theoretic operations (union, inter, diff): `vec \ chr = raw | lg1 | int | dbl | clx`
Usual set properties (e.g. De Morgan laws): `~(t1 | t2) = ~t1 & ~t2`

Implementation



Examples

```
1 rescale <- function(x: dbl, na.rm: lg11 = TRUE) -> dbl {
2   (x - min(x, na.rm=na.rm)) / (max(x, na.rm=na.rm) - min(x, na.rm=na.rm))
3 }
4 iris: { Sepal.Length: dbl, «3 more», Species: int<factor> }<data.frame>
5 widths: dbl <- iris$Sepal.Width
6 r: dbl <- rescale(widths)
7 species: int<factor> <- iris$Species
8 rescale(species)
```

Type error argument `x` expects `dbl`, but `species` is `int<factor>` — a factor is disjoint from `dbl`.
`typr(shape-mismatch) · rescale.R [8, 9]`

R expression	Type	Comment
<code>NULL</code>	<code>null</code>	its own type, distinct from empty vectors
<code>list(1, 2, 3)</code>	<code>{ dbl }</code>	homogeneous list of doubles
<code>list(1, "a", TRUE)</code>	<code>{ dbl chr lg1 }</code>	heterogeneous — union element type
<code>list(x = 1, y = 2)</code>	<code>{ x : dbl, y : dbl }</code>	record with named fields
<code>list(x = 1, y = "a", 3)</code>	<code>{ x : dbl, y : chr, any }</code>	named fields plus anything else
<code>factor(c("a", "b", "a"))</code>	<code>int<factor></code>	codes are int; class makes it disjoint from <code>dbl</code>
<code>ordered(c("lo", "hi"))</code>	<code>int<ordered, factor ...></code>	carries two classes at once
<code>matrix(1:6, 2, 3)</code>	<code>int<matrix, array ...></code>	shape is the type bit R hides
<code>data.frame(id = 1:3, name = c("a", "b", "c"))</code>	<code>{ id : int, name : chr }<data.frame ...></code>	list shape plus S3 class together
<code>lm(y ~ x)</code>	<code>{ coefficients : dbl, residuals : dbl, any }<lm></code>	a big record with a class
<code>function(x) x + 1</code>	<code>(x: dbl)->dbl & (x: clx)->clx & (x: clx1)->clx1</code>	increment function
<code>nrow</code>	<code>(x: vec <array ...> <data.frame ...>) -> int</code>	<code>nrow</code> function – can take vectors, arrays, <code>df</code>
<code>df\$col</code>	<code>(x: { #col:'a ...'}<data.frame ...>, #col) -> 'a</code>	extraction of a dataframe column

Subtyping

Type of <code>c(...)</code>	raw	logical	integer	double	complex	character
raw	raw	logical	integer	double	complex	character
logical	logical	logical	integer	double	complex	character
integer	integer	integer	integer	double	complex	character
double	double	double	double	double	complex	character
complex	complex	complex	complex	complex	complex	character
character	character	character	character	character	character	character

`raw1 < lg11 < int1 < dbl1 < clx1 < chr1`
`raw < lg1 < int < dbl < clx < chr`

```
df : { id:dbl ...}<data.frame ...>
df <- data.frame(id = 1L:3L) # OK, id is int < dbl
df <- data.frame(id = 1:3, name = c("a","b","c")) # OK, type of df allows extra columns
df <- data.frame(id = c("a","b","c")) # NOT OK, id is chr > dbl
```

Function polymorphism

Overloading (aka ad-hoc polymorphism)

```
typeof : ( x : { any } ) -> "list" rbind : ( ... : <data.frame ...> ) -> <data.frame ...>
typeof : ( x : null ) -> "NULL" rbind : ( ... : <matrix ...> ) -> <matrix ...>
typeof : ( x : lg1 ) -> "logical" rbind : ( ... : vec<> ) -> <matrix>
```

Genericity (aka parametric polymorphism)

```
identity : (x : 'a) -> 'a (+) : (x : int, y : int) -> int
[.] : ({'a}, dbl) -> 'a (+) : (x : dbl, y : dbl) -> dbl
[.] : (v('a), dbl) -> v1('a) (+) : (x : chr, y : chr) -> chr
```

Type variables (`'a`, `'b`): capture a single type. Row variables (``r`): capture a list of types.

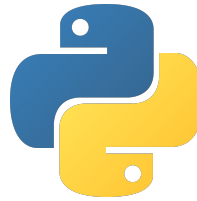
```
lapply : (X : v('a), FUN : @(v1('a), ... : `r) -> 'b, ... : `r) -> {'b}
lapply : (X : {'a}, FUN : @( 'a , ... : `r) -> 'b, ... : `r) -> {'b}
```

Type narrowing

```
neg <- function(x: null | lg1 | dbl) -> null | lg1 | dbl {
  if (typeof(x) == "NULL") return(NULL)
  if (typeof(x) == "logical") return(!x)
  return(-x)
}
```

The rest is typed under the assumption that: `x : ~null`

Related work



- **Official syntax** for type specifications, but **no official type-checker** implementation
- Many type-checkers co-exist: MyPy, Pyright, Ty
- Mostly **incompatible** as types are not formalized



- **No official syntax** for types
- Standard way is assertions (`stopifnot`)
- Packages providing type annotations checked at **runtime**: types, rtypes



- Statically-typed variant of JS, **widely adopted**
- A **new syntax** for types on top of JS
- Type annotations are **erased**, yielding valid JS



- Started **retrofitting** gradual types 5 years ago
- No type annotations yet, but it is planned
- **Based on set-theoretic types, as our work!**